EXPERIMENT 10

MINI PROJECT REPORT

End-to-End DevOps Pipeline for the MedBot AI Medical Chatbot

Subject: Agile Software Development and DevOps

1. Aim

To design and implement a complete Agile and DevOps workflow for a web application: plan the work in Jira, manage the code in Git and GitHub, package the application with Docker, test it with a scripted multi-turn test suite, deploy it to the cloud (Render and Hugging Face Spaces), and monitor it through a health endpoint and application logs.

2. Objectives

- Manage the project backlog and sprints using Agile (Scrum) practices in Jira.
- Keep all source code under version control in a GitHub repository.
- Containerise the application with a Dockerfile so it runs the same on a laptop and in the cloud.
- Verify the chatbot with automated multi-turn conversation tests.
- Deploy the service using a Render blueprint (render.yaml) and a Docker based Hugging Face Space.
- Observe the running system through the /health endpoint and structured logging.

3. Introduction

MedBot AI is an AI medical assistant built with FastAPI. A user describes symptoms or asks a health question in a browser chat window; the back end remembers the conversation, extracts structured patient information (age, symptoms, duration, medications and so on), retrieves relevant background knowledge from a Supabase vector store (Retrieval Augmented Generation), and asks a Groq hosted large language model to write a careful, personalised answer. The system prompt forbids definitive diagnoses and contains an emergency recognition layer that tells the user to seek urgent care for red flag symptoms such as severe chest pain or difficulty breathing.

The purpose of this experiment is not only the application itself but the engineering process around it. Every change moves along the same path: a Jira ticket, a Git commit, a build of the Docker image, an automated deployment, and live checking through the health endpoint. This shortens feedback time and keeps releases repeatable, which is the central idea of DevOps.

4. Tools and Technologies

Item	Details
Student	Aaditya Surve (Roll no. 2401119), Batch T22
Project	MedBot AI – AI-powered medical chatbot (RAG)
Source repository	GitHub (MedBot AI project)
DevOps tools	Jira, Git / GitHub, Docker, Render, Hugging Face Spaces
Application stack	FastAPI, Uvicorn, LangChain, Supabase (pgvector), Groq, HuggingFace embeddings

Item	Details
Date	15 October 2026

Tool	Purpose	Usage in this project
Jira	Agile planning, backlog, sprints	Backlog and sprint tickets for each feature
Git / GitHub	Version control and collaboration	Repository holding backend, frontend, Dockerfile and render.yaml
Docker	Containerisation	Dockerfile based on python:3.10-slim, runs Uvicorn on port 7860
Render	Cloud hosting (CD)	render.yaml blueprint builds and starts the web service
Hugging Face Spaces	Alternative hosting	Docker Space built from the same Dockerfile
FastAPI + Uvicorn	Web framework and server	Serves the front end and the /chat, /health, /clear APIs
Supabase	Vector database	Stores document embeddings; match_documents similarity search
Groq (ChatGroq)	LLM inference	Model openai/gpt-oss-20b, temperature 0.2
HuggingFace embeddings	Text embeddings	all-MiniLM-L6-v2 running locally, no API key needed

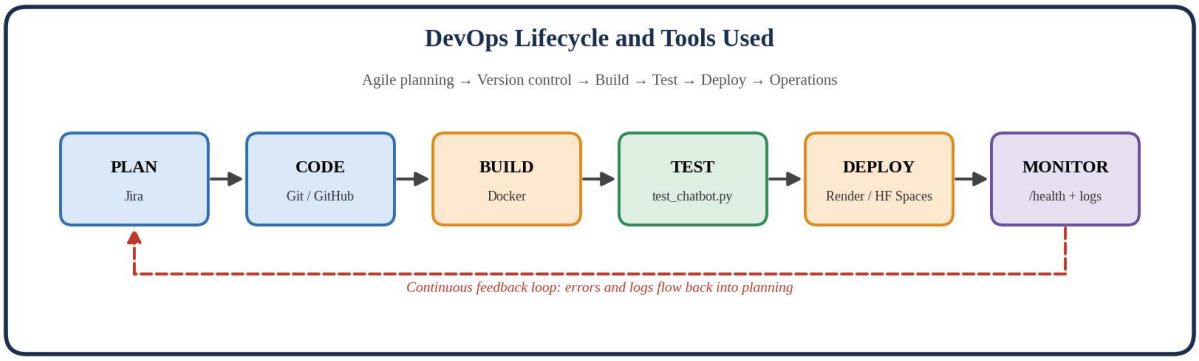


Figure 1: DevOps lifecycle mapped to the tools used

5. Agile Planning with Jira

The project followed Scrum. The product backlog was written first and broken into small tickets, each describing one deliverable. Work was pulled into short sprints, and every Git commit message can start with its ticket number so that a code change can be traced back to a requirement. The table lists the main work items that the repository contains.

No.	Work item	Area
1	Project skeleton: FastAPI back end serving a static HTML/CSS/JS front end	Setup
2	Supabase vector store and data ingestion script (ingest.py)	Data / RAG
3	Groq LLM integration and /chat API	Back end
4	Conversation memory and patient context extraction	Back end

No.	Work item	Area
5	Modular prompts with an emergency safety layer	Prompts
6	Chat interface with suggestion chips and Markdown rendering	Front end
7	Dockerfile and Render blueprint	Deployment
8	Multi-turn conversation tests (test_chatbot.py)	Testing

6. System Architecture

The application is a single FastAPI service. The browser loads the static front end from the service, which then sends each message to the /chat endpoint. The back end keeps per-session memory, retrieves background documents from Supabase using locally computed embeddings, and calls the Groq model to produce the reply. Secrets (Supabase and Groq keys) are supplied as environment variables and are never exposed by the API.

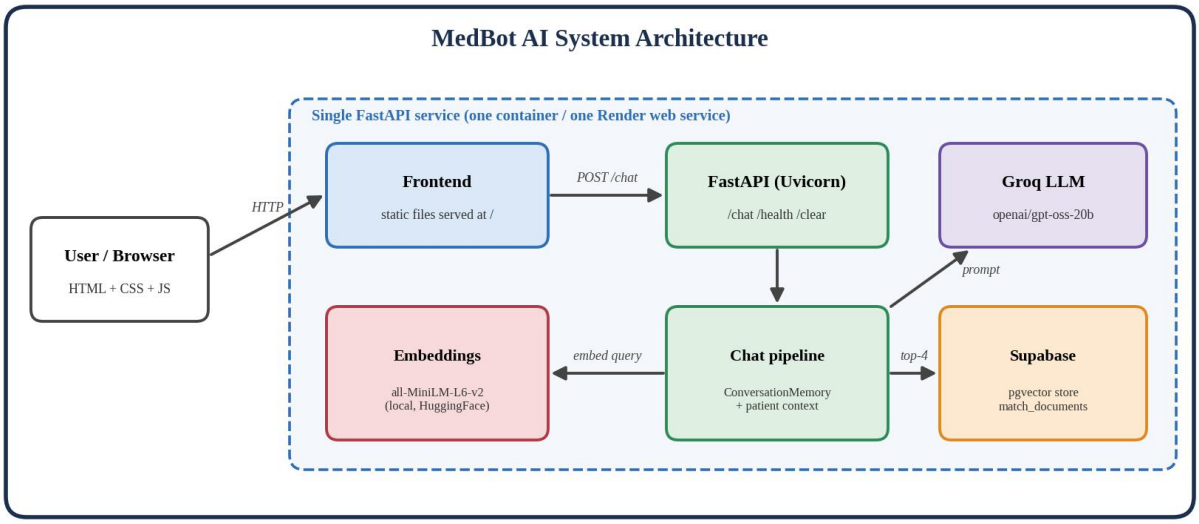


Figure 2: System architecture of MedBot AI

The repository is organised into a backend folder (Python modules and requirements), a frontend folder (static files) and the deployment files at the root.

```
user@medbot: ~/Medbot-main
$ find . -type f | sort
./.gitignore
./Dockerfile
./README.md
./backend/.python-version
./backend/context_manager.py
./backend/ingest.py
./backend/main.py
./backend/prompts.py
./backend/requirements.txt
./backend/runtime.txt
./backend/test_chatbot.py
./frontend/favicon.svg
./frontend/index.html
./frontend/script.js
./frontend/style.css
./render.yaml
```

Figure 3: Project file structure

6.1 API Endpoints

FastAPI also publishes interactive Swagger documentation at /docs. The routes defined in main.py are listed below.

Method	Endpoint	Purpose
GET	/	Serves the chat front end (index.html)
GET	/health	Health check: status, which env vars are set, number of active sessions
POST	/chat	Body {message, session_id}; returns {response, session_id}
POST	/clear	Deletes a session's history and patient context
GET	/static/*	Serves style.css and script.js from the frontend folder
GET	/docs	Auto-generated Swagger UI

7. Implementation

This section shows the main parts of the source code. The back end is split into four modules: main.py (server and pipeline), context_manager.py (memory and patient context), prompts.py (prompt building and RAG decision) and ingest.py (loading data into Supabase).

7.1 Back-end setup

main.py connects to Supabase, loads the local embedding model, creates the vector store retriever (top 4 documents) and the Groq LLM client. API keys are read from environment variables with python-dotenv.

```python
# ──────────────────────────────────────────────
# 1. Supabase
# ──────────────────────────────────────────────
supabase_url = os.environ.get("SUPABASE_URL")
supabase_key = os.environ.get("SUPABASE_SERVICE_KEY")
supabase: Client = create_client(supabase_url, supabase_key)

# ──────────────────────────────────────────────
# 2. Embeddings (local, no API key required)
# ──────────────────────────────────────────────
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# ──────────────────────────────────────────────
# 3. Vector store / retriever
# ──────────────────────────────────────────────
vector_store = SupabaseVectorStore(
    client=supabase,
    embedding=embeddings,
    table_name="documents",
    query_name="match_documents",
)
retriever = vector_store.as_retriever(search_kwargs={"k": 4})

# ──────────────────────────────────────────────
# 4. LLM (Groq)
# ──────────────────────────────────────────────
llm = ChatGroq(
    temperature=0.2,          # slight warmth for natural language
    model_name="openai/gpt-oss-20b",
    groq_api_key=os.environ.get("GROQ_API_KEY"),
)
```

Figure 4: Supabase, embeddings, retriever and LLM set-up in main.py

7.2 Chat pipeline

The function _run_chat executes the same seven steps for every user message. Retrieved documents are appended to the human message instead of the system prompt, so they stay tied to the current question.

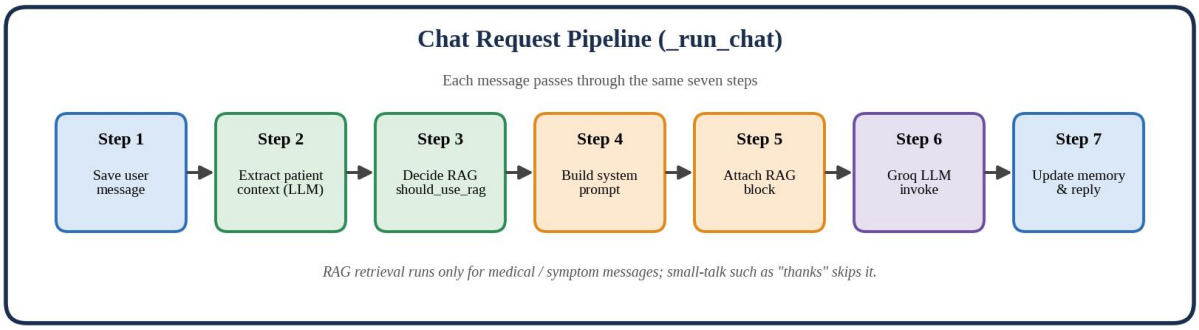


Figure 5: Seven-step chat request pipeline

```python
# ─────────────────────────────────────────────
# 9. Core chat pipeline
# ─────────────────────────────────────────────
def _run_chat(session: dict, user_message: str) -> str:
    """Execute the full conversation pipeline and return the assistant reply."""
    memory: ConversationMemory = session["memory"]
    patient_context: dict = session["patient_context"]

    # Step 1 — Add user message to memory
    memory.add_user(user_message)

    # Step 2 — Extract patient context from the new message
    history_for_extraction = memory.get_history(exclude_last_user=True)
    session["patient_context"] = update_patient_context(
        patient_context,
        user_message,
        history_for_extraction,
        llm,
    )
    patient_context = session["patient_context"]

    # Step 3 — Decide whether to retrieve from vector store
    history_for_prompt = memory.get_history(exclude_last_user=True)
    rag_docs = ""
    if should_use_rag(user_message, patient_context):
        rag_query = build_rag_query(user_message, patient_context, history_for_prompt)
        logger.info("RAG query: %s", rag_query[:120])
        rag_docs = _retrieve_docs(rag_query)

    # Step 4 — Build the system prompt
    system_prompt_text = build_system_prompt(patient_context, history_for_prompt)

    # Step 5 — Append RAG block to the human message (not system prompt)
    #          so it is clearly associated with the current question
    rag_block = build_rag_block(rag_docs)
    human_content = user_message
    if rag_block:
        human_content = f"{user_message}\n\n{rag_block}"

    # Step 6 — Call LLM
    messages = [
        SystemMessage(content=system_prompt_text),
        HumanMessage(content=human_content),
    ]
    response = llm.invoke(messages)
    assistant_reply = response.content if hasattr(response, "content") else str(response)

    # Step 7 — Update memory with assistant reply
    memory.add_assistant(assistant_reply)

    return assistant_reply
```

Figure 6: The _run_chat function in main.py

7.3 API routes

The routes serve the front end, expose a health check, run the chat pipeline and clear sessions. Errors in the pipeline are logged on the server and a friendly message is returned, so internal details are never shown to the user.

```python
# ─────────────────────────────────────────────
# 10. Routes
# ─────────────────────────────────────────────
@app.get("/")
async def serve_frontend():
    return FileResponse(os.path.join(FRONTEND_DIR, "index.html"))


@app.get("/health")
async def health_check():
    """Lightweight health check — never exposes secrets."""
    return {
        "status": "ok",
        "env_vars": {
            "SUPABASE_URL": bool(os.environ.get("SUPABASE_URL")),
            "SUPABASE_SERVICE_KEY": bool(os.environ.get("SUPABASE_SERVICE_KEY")),
            "GROQ_API_KEY": bool(os.environ.get("GROQ_API_KEY")),
        },
        "active_sessions": len(_sessions),
    }


@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    # Assign a session id if the frontend didn't provide one
    session_id = request.session_id.strip() or str(uuid.uuid4())

    try:
        session = _get_or_create_session(session_id)
        reply = _run_chat(session, request.message.strip())
        return ChatResponse(response=reply, session_id=session_id)
    except Exception as exc:
        logger.error("Chat error for session %s: %s", session_id, exc, exc_info=True)
        # Return a user-friendly error without exposing internals
        return ChatResponse(
            response=(
                "I'm sorry, I encountered a technical issue and couldn't process "
                "your message. Please try again, or refresh the page to start a "
                "new conversation."
            ),
            session_id=session_id,
        )
```

Figure 7: /, /health and /chat routes in main.py

7.4 Patient context and conversation memory

After every message a small LLM call extracts only explicitly stated facts into a JSON object. The helper below merges new facts into the stored context: lists are united without duplicates and scalar values are overwritten only by real values. ConversationMemory keeps a sliding window of the last 10 exchanges to stay within token limits.

```python
def _merge_context(existing: dict, updates: dict) -> dict:
    """Merge extracted updates into the existing patient context.

    Lists are unioned (no duplicates).
    Scalar fields are overwritten only when a new non-null value is provided.
    """
    import copy
    merged = copy.deepcopy(existing)

    for key, new_val in updates.items():
        if key not in merged:
            continue  # ignore unknown keys

        existing_val = merged[key]

        if isinstance(existing_val, list):
            if isinstance(new_val, list) and new_val:
                # Add new items, preserving order and avoiding duplicates
                seen = {v.lower() if isinstance(v, str) else v for v in existing_val}
                for item in new_val:
                    item_key = item.lower() if isinstance(item, str) else item
                    if item_key not in seen:
                        existing_val.append(item)
                        seen.add(item_key)
        else:
            # Scalar — only overwrite with a real value
            if new_val is not None and new_val != "":
                merged[key] = new_val

    return merged
```

Figure 8: _merge_context in context_manager.py

7.5 Deciding when to use RAG

Retrieval is skipped for small talk such as “thanks” or a bare age introduction, and used whenever the message contains medical keywords or the patient context already holds symptoms. This saves a vector search on messages that cannot benefit from it.

```python
def should_use_rag(message: str, patient_context: dict) -> bool:
    """Decide if RAG retrieval is likely to add value for this message.

    Returns False for purely conversational messages.
    Returns True whenever the message has medical/symptom content.
    """
    import re

    msg_lower = message.lower().strip()

    for pattern in _NON_MEDICAL_PATTERNS:
        if re.match(pattern, msg_lower, re.IGNORECASE):
            return False

    medical_keywords = [
        "pain", "ache", "hurt", "symptom", "fever", "cough", "cold", "flu",
        "headache", "nausea", "dizzy", "tired", "fatigue", "rash", "bleed",
        "breath", "chest", "stomach", "back", "throat", "ear", "eye", "skin",
        "allerg", "medic", "drug", "dose", "doctor", "hospital", "treat",
        "diagnos", "condition", "disease", "infection", "inflam", "swollen",
        "blood", "heart", "pressure", "sugar", "diabet", "cancer", "sleep",
        "anxiety", "depress", "mental", "stress", "weight", "diet", "vitamin",
        "supplement", "prescri", "side effect", "interact",
    ]
    has_medical = any(kw in msg_lower for kw in medical_keywords)
    has_context_symptoms = bool(patient_context.get("symptoms"))
```

Figure 9: should_use_rag in prompts.py

7.6 Loading the knowledge base

ingest.py splits text into chunks, embeds them with all-MiniLM-L6-v2 and stores them in the Supabase documents table. It is run once before the first start.

```python
import os
from dotenv import load_dotenv
from supabase.client import Client, create_client
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import SupabaseVectorStore
from langchain_text_splitters import CharacterTextSplitter

load_dotenv()

# 1. Setup Supabase
supabase_url = os.environ.get("SUPABASE_URL")
supabase_key = os.environ.get("SUPABASE_SERVICE_KEY")
supabase: Client = create_client(supabase_url, supabase_key)

# 2. Setup FREE Hugging Face Embeddings
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# 3. The text we want the AI to learn
text_data = "The capital of France is Paris. It is known for the Eiffel Tower. Aspirin is used to treat pain and fever."

# 4. Split the text into manageable chunks
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
docs = text_splitter.create_documents([text_data])

# 5. Push the vectors into your Supabase library
vector_store = SupabaseVectorStore.from_documents(
    docs,
    embeddings,
    client=supabase,
    table_name="documents",
    query_name="match_documents",
)
```

Figure 10: ingest.py – pushing documents into Supabase

7.7 Front end

script.js keeps a stable session id for the browser tab, posts each message to /chat and renders the Markdown reply. The BACKEND_URL constant lets the same front end talk to a Render deployment.

```
async function sendMessage() {
    const text = messageInput.value.trim();
    if (!text) return;

    // Hide welcome screen
    const welcome = document.getElementById('welcomeContainer');
    if (welcome) welcome.remove();

    // Add user message
    appendMessage('user', text);
    messageInput.value = '';
    setLoading(true);

    // Show typing indicator
    const typingEl = showTypingIndicator();

    try {
        // Fix URL dynamically in case of trailing `/chat`
        const url = BACKEND_URL
            ? (BACKEND_URL.endsWith('/chat') ? BACKEND_URL : `${BACKEND_URL}/chat`)
            : '/chat';
        const res = await fetch(url, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ message: text, session_id: SESSION_ID })
        });
        const data = await res.json();
        // Keep the canonical session_id from the server
        if (data.session_id) {
            SESSION_ID = data.session_id;
            sessionStorage.setItem('medbot_session_id', SESSION_ID);
        }
        typingEl.remove();
        appendMessage('bot', data.response);
    } catch (err) {
        typingEl.remove();
        appendMessage('bot', 'Sorry, something went wrong. Please check the server.');
    } finally {
        setLoading(false);
```

Figure 11: sendMessage in script.js

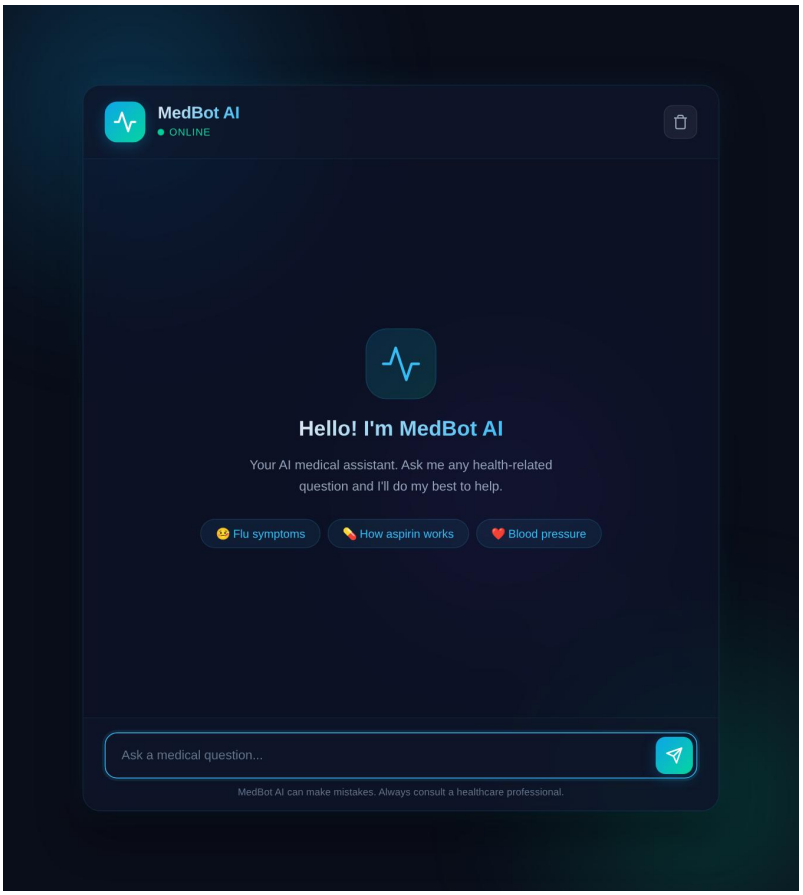


Figure 12: MedBot AI welcome screen (frontend rendered from index.html, style.css and script.js)

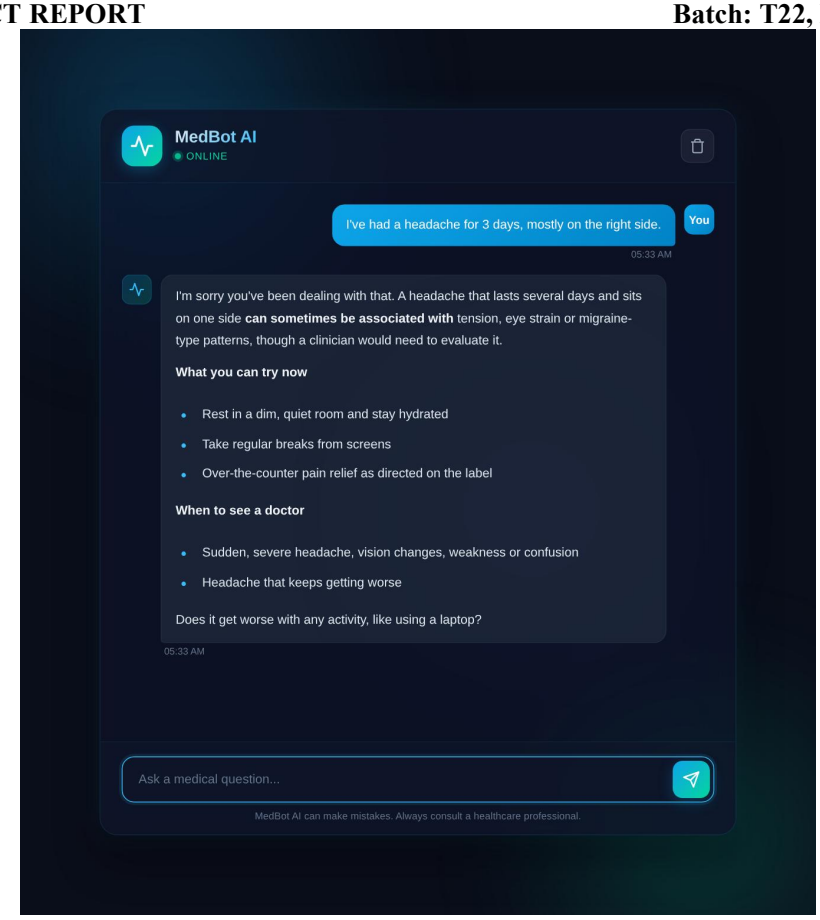


Figure 13: MedBot AI chat window with an example conversation (sample text, rendered with the real frontend)

8. Containerisation with Docker

The whole application is described by one Dockerfile, so the environment is identical on a laptop, on Render and on Hugging Face Spaces. The image uses python:3.10-slim, creates a non-root user (uid 1000, required by Hugging Face), copies the project, installs the pinned dependencies from backend/requirements.txt and starts Uvicorn on port 7860.

```
# Use a lightweight Python 3.10 image
FROM python:3.10-slim

# Create a user to avoid permission issues on HF
RUN useradd -m -u 1000 user
USER user
ENV PATH="/home/user/.local/bin:$PATH"
WORKDIR /home/user/app

# Copy your whole project into the container
COPY --chown=user . .

# Install your backend dependencies
RUN pip install --no-cache-dir --upgrade -r backend/requirements.txt

# Tell HF we are using port 7860 (this is mandatory)
EXPOSE 7860
```

Figure 14: Dockerfile of the project

9. Testing

test_chatbot.py is an end-to-end test script that needs the server and API keys. It starts a new session for each scenario, sends a sequence of user messages to the running /chat endpoint and prints the first

500 characters of every reply, so context memory, clarifying questions and emergency handling can be checked quickly after each change.

Test	Messages	What it checks
1 Context continuity	3	Headache follow-ups keep earlier details (3 days, right side, laptop)
2 Age memory	2	Age given first is remembered when a fever is described
3 Clarifying questions	1	Vague message “my stomach hurts” leads to targeted questions
4 Context-aware RAG	2	Fatigue plus dizziness are combined into one retrieval query
5 Emergency red flag	1	Chest pain and breathing trouble trigger an urgent-care warning
6 Context switch	3	Resolved headache is separated from a new knee problem

```
    r = requests.post(BASE, json={"message": msg, "session_id": session_id}, timeout=30)
    d = r.json()
    return d["response"]


def run_test(name: str, turns: list[str]) -> None:
    session_id = str(uuid.uuid4())
    print("\n" + "=" * 70)
    print(f"  {name}")
    print("=" * 70)
    for msg in turns:
        print(f"\n  USER: {msg}")
        reply = chat(msg, session_id)
        # Print first 500 chars of each response
        short = reply[:500] + ("..." if len(reply) > 500 else "")
        print(f"  BOT : {short}")
    print()


if __name__ == "__main__":
    # Test 1 — Basic context continuity
    run_test("TEST 1: Context continuity (headache)", [
        "I've had a headache for 3 days.",
        "It's mostly on the right side.",
        "It gets worse when I use my laptop.",
    ])

    # Test 2 — Personal information remembered
    run_test("TEST 2: Age memory (fever)", [
```

Figure 15: run_test helper and the first test scenarios in test_chatbot.py

9.1 Logic checks run locally

The pure-Python parts of the pipeline do not need API keys, so they were also run directly: the RAG decision, context merging, the RAG query builder, the sliding-window memory and the system prompt assembly. The output below shows the results. Greetings and bare introductions skip retrieval, repeated symptoms are merged without duplicates (headache and Headache become one entry), and the memory window keeps only the last two exchanges.

```
user@medbot: ~/Medbot-main/backend

$ cd backend && python checks.py
== should_use_rag ==
'thanks'                              -> False
"I'm 20 years old."                   -> False
"I've had a headache for 3 days."     -> True
'How does aspirin work?'              -> False
'ok'                                  -> False

== _merge_context ==
age              : 20
symptoms         : ['headache', 'nausea']
symptom_duration : 3 days
location         : right temple

== build_rag_query ==
it gets worse with my laptop symptoms: headache, nausea duration: 3 days I have a headache

== ConversationMemory (window=2 turns) ==
['q3', 'a3', 'q4', 'a4']

== system prompt sections ==
length: 4072 chars; safety layer present: True ; patient context present: True
```

Figure 16: Output of the local logic checks

The check also showed a limitation: “How does aspirin work?” does not trigger retrieval, because should_use_rag relies on a fixed keyword list that contains words such as medic and drug but no individual drug names. This is listed under Future Scope.

10. Continuous Delivery and Deployment

Delivery is automated by the hosting platforms. When a change is pushed to GitHub, Render reads render.yaml, installs the requirements from the backend folder and starts the service with Uvicorn on the port that Render provides. The three secret values are declared with sync: false, so they are entered once in the Render dashboard and never committed to Git. The same repository also builds as a Docker Space on Hugging Face because of the Dockerfile.

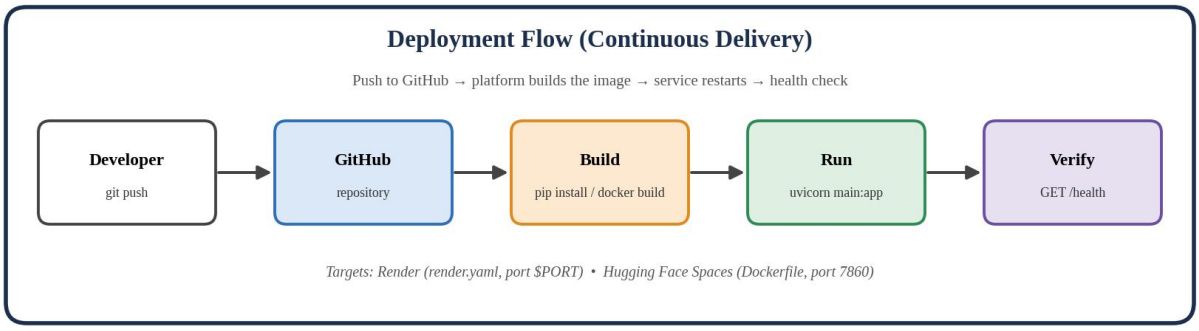


Figure 17: Deployment flow from commit to running service

```yaml
services:
  - type: web
    name: medbot-ai
    runtime: python
    rootDir: backend
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_SERVICE_KEY
        sync: false
      - key: GROQ_API_KEY
        sync: false
      - key: HF_API_KEY
        sync: false
```

Figure 18: render.yaml blueprint

11. Monitoring

The /health endpoint (Figure 7) returns the service status, a true/false flag for each required environment variable and the number of active sessions. It never reveals secret values, so it can be used safely by Render's health check or by hand after every deployment. Python logging at INFO level records the RAG query for each message, retrieval failures and chat errors with stack traces.

12. Results

Area	Result
Planning	Backlog and sprint tickets planned in Jira
Source control	Source code hosted on GitHub
Code base	About 2,000 lines: 4 back-end modules, 1 test script, HTML/CSS/JS front end
Containers	1 Dockerfile; image runs the FastAPI service on port 7860
Testing	6 multi-turn scenarios with 12 messages (memory, RAG, emergencies) plus local checks of the prompt and context logic
Deployment	Render blueprint and Hugging Face Docker Space
Monitoring	/health endpoint and INFO level logging

13. Challenges and Learnings

- Dependency compatibility: supabase, gotrue, httpx and httpcore are pinned to exact versions in requirements.txt because mismatched releases break the Supabase client; pinning keeps builds repeatable.
- Configuration errors: a space after = in the .env file or a revoked key causes Invalid API Key errors. The /health endpoint reports which variables are set so this is found quickly.
- Unreliable LLM output: the context extractor must return JSON, but models sometimes add extra text. A fallback parser and a try/except keep the chat working even when extraction fails.
- Platform differences: the same code had to run locally, on Render (dynamic $PORT) and on Hugging Face (port 7860), which is why the server reads the port from the environment and why a Dockerfile was added.
- Keyword based RAG gate: running should_use_rag showed that a plain drug question such as "How does aspirin work?" skips retrieval because no keyword matches. Testing individual functions found this quickly.

- Session state: conversations are kept in memory, so they disappear on restart and cannot be shared across several instances.
- Pipeline as code: keeping the Dockerfile and render.yaml in Git makes the set-up repeatable and reviewable.

14. Conclusion

The mini project shows an Agile and DevOps cycle for a real AI application. Work is planned in Jira, kept in GitHub, packaged with Docker, verified by a scripted test suite, delivered to Render and Hugging Face Spaces, and observed through a health endpoint and logs. The result is a faster, more repeatable and more transparent release process than manual deployment.

Future Scope

- Replace the keyword list in should_use_rag with a better medical intent check so that drug and condition names also trigger retrieval.
- Add a GitHub Actions workflow that lints the code, builds the Docker image and runs the tests on every pull request.
- Add Prometheus metrics and a Grafana dashboard for request rate and latency.
- Store sessions in Redis or a database instead of process memory.
- Restrict CORS origins and add rate limiting and authentication.
- Ingest a real, reviewed medical knowledge base; ingest.py currently loads only a short sample text.
- Add Jenkins or GitHub Actions based deployment with automated rollback.

References

- FastAPI documentation, fastapi.tiangolo.com
- LangChain documentation, python.langchain.com
- Supabase vector store documentation, supabase.com/docs
- Groq API documentation, console.groq.com/docs
- Docker documentation, docs.docker.com
- Render blueprint (render.yaml) documentation, render.com/docs
- Hugging Face Docker Spaces documentation, huggingface.co/docs/hub/spaces-sdks-docker
- Atlassian Jira Scrum guide, atlassian.com/agile/scrum